

Skills & Rules

Customizing AI Agents with Claude Code and Cursor

VibeCoding

February 28, 2026

Part 1: Skills

- What are Skills?
- How Skills Work
- Skill Directories
- SKILL.md File Format
- Advanced Features
- Using Skills

Part 2: Rules

- Cursor Rules
- Project Rules
- User Rules
- AGENTS.md
- CLAUDE.md Files
- Skills vs Rules Comparison

What are Skills?

Definition

A **skill** is a portable, version-controlled package that teaches agents how to perform domain-specific tasks.

Skills can include:

- Scripts
- Templates
- References that agents act on using their tools

Key Characteristics of Skills

Portable

Skills work across any agent that supports the Agent Skills standard.

Version-controlled

Skills are stored as files and can be tracked in your repository, or installed via GitHub repository links.

Actionable

Skills can include scripts, templates, and references that agents act on using their tools.

Progressive

Skills load resources on demand, keeping context usage efficient.

Skill Discovery & Invocation

When Cursor/Claude Code starts:

- 1 It automatically discovers skills from skill directories
- 2 Makes them available to Agent
- 3 Agent decides when skills are relevant based on context

Manual Invocation (Claude Code):

- Type `/skill` or `/` to view available skills
- Select a skill from the list to invoke it

Manual Invocation (Cursor):

- Type `/` in Agent chat
- Search for the skill name

Skill Loading Locations

Claude Code:

Location	Scope
<code>.claude/skills/</code>	Project-level
<code>~/.claude/skills/</code>	User-level (global)

Cursor:

Location	Scope
<code>.agents/skills/</code>	Project-level
<code>.cursor/skills/</code>	Project-level
<code>~/.cursor/skills/</code>	User-level (global)

Note: For compatibility, Cursor also loads skills from Claude and Codex directories.

Skill Directory Structure

Claude Code (Project):

```
.claude/  
|-- skills/  
    |-- my-skill/  
        |-- SKILL.md  
+-- deploy-app/  
    |-- SKILL.md  
+-- scripts/
```

Cursor (Project):

```
.cursor/  
|-- skills/  
    |-- my-skill/  
        |-- SKILL.md  
+-- deploy-app/  
    |-- SKILL.md  
+-- scripts/
```

Global skills: ~/.claude/skills/ or ~/.cursor/skills/

SKILL.md Frontmatter

Each skill is defined with YAML frontmatter:

```
---  
name: my-skill  
description: Short description of what this skill does  
            and when to use it.  
---  
# My Skill  
Detailed instructions for the agent.
```

Best Practices:

- Include *When to Use* section
- Include *Instructions* with step-by-step guidance
- Use the ask questions tool if you need to clarify requirements

Frontmatter Fields

Field	Required	Description
name	Yes	Skill identifier. Lowercase letters, numbers, and hyphens only.
description	Yes	Describes what the skill does and when to use it.
license	No	License name or reference to a bundled license file.
compatibility	No	Environment requirements.
metadata	No	Arbitrary key-value mapping for additional metadata.
disable-model-invocation	No	When true, skill only runs when explicitly invoked.

Disabling Automatic Invocation

Default Behavior

Skills are automatically applied when the agent determines they are relevant.

Manual-Only Mode

Set `disable-model-invocation: true` to make a skill behave like a traditional slash command.

Use case: When you want full control over when a skill is used.

Including Scripts in Skills

Skills can include executable code in the `scripts/` directory:

```
---  
name: deploy-app  
description: Deploy the application to staging or production.  
---  
# Deploy App  
## Usage  
Run: 'scripts/deploy.sh <environment>'  
  
## Pre-deployment Validation  
Run: 'python scripts/validate.py'
```

Tip: Scripts should be self-contained with helpful error messages.

Optional Directories

Directory	Purpose
scripts/	Executable code that agents can run
references/	Additional documentation loaded on demand
assets/	Static resources like templates, images, or data files

Best Practice

Keep your main `SKILL.md` focused and move detailed reference material to separate files for efficient context usage.

Installing Skills from GitHub

- 1 Open Cursor Settings → Rules
- 2 In the Project Rules section, click **Add Rule**
- 3 Select **Remote Rule (Github)**
- 4 Enter the GitHub repository URL

This allows sharing skills across teams and projects!

What are Cursor Rules?

Definition

Rules provide system-level instructions to Agent. They bundle prompts, scripts, and more together, making it easy to manage and share workflows across your team.

Four Types of Rules:

- **Project Rules** – Stored in `.cursor/rules`, version-controlled
- **User Rules** – Global to your Cursor environment
- **Team Rules** – Team-wide rules from the dashboard
- **AGENTS.md** – Simple markdown instructions

How Rules Work

The Problem:

- LLMs don't retain memory between completions

The Solution:

- Rules provide persistent, reusable context at the prompt level
- When applied, rule contents are included at the start of the model context
- This gives the AI consistent guidance for generating code

Key Difference

Rules are loaded into context every time, while Skills load resources on-demand for efficiency.

Project Rules Structure

Project rules live in `.cursor/rules` as markdown files:

```
.cursor/rules/  
  react-patterns.mdc      # Rule with frontmatter  
  api-guidelines.md      # Simple markdown rule  
  frontend/  
    components.md        # Organize rules in folders
```

Use Project Rules to:

- Encode domain-specific knowledge about your codebase
- Automate project-specific workflows or templates
- Standardize style or architecture decisions

Rule File Format

Each rule is a markdown file with frontmatter metadata:

```
---  
description: "Standards for frontend components"  
globs: ["src/**/*.ts", "src/**/*.tsx"]  
alwaysApply: false  
---  
# Frontend Component Standards  
- Use functional components with hooks  
- Prefer composition over inheritance  
- Follow the existing component patterns
```

globs: Specifies which files this rule applies to using glob patterns.

Understanding Globs

Glob patterns determine when a rule is automatically applied:

Pattern	Matches
"*.ts"	All .ts files in current directory
"**/*.ts"	All .ts files in any subdirectory
"src/**/*.ts,tsx"	.ts and .tsx files under src/
"components/**/*.tsx"	.tsx files under components/
"*.js,ts,jsx,tsx" Files with any of these extensions	

Example: When you edit `src/components/Button.tsx`, rules with globs: `["src/**/*.ts,tsx"]` are automatically applied!

Rule Types

Type	When Applied
Always Apply	Every chat session
Apply Intelligently	When Agent decides it's relevant
Apply to Specific Files	When file matches a pattern
Apply Manually	When @-mentioned in chat

User Rules

Definition:

- Global preferences defined in Cursor Settings → Rules
- Apply across all projects
- Used by Agent (Chat)

Best Use Cases:

- Setting preferred communication style
- Defining personal coding conventions
- Global workflow preferences

Example

"Please reply in a concise style. Avoid unnecessary repetition or filler language."

Definition:

- Simple markdown file for defining agent instructions
- Place in project root as alternative to `.cursor/rules`
- No metadata or complex configurations

AGENTS.md Example

Project Instructions

Code Style

- Use TypeScript for all new files
- Prefer functional components in React
- Use snake_case for database columns

Architecture

- Follow the repository pattern
- Keep business logic in service layers

Cursor supports AGENTS.md in project root and subdirectories.

Best Practices for Rules

Keep Rules Focused

- Keep rules under 500 lines
- Split large rules into multiple, composable rules
- Provide concrete examples or referenced files

What to AVOID

- Copying entire style guides (use a linter instead)
- Documenting every possible command
- Duplicating what's already in your codebase

Rule Management Tips

Start Simple:

- Add rules only when you notice Agent making the same mistake repeatedly
- Don't over-optimize before you understand your patterns

Version Control:

- Check your rules into git so your whole team benefits
- Update rules when you see Agent make mistakes

Reference Files:

- Reference files instead of copying their contents
- This keeps rules short and prevents staleness

Skills vs Rules: Key Differences

Skills	Rules (Cursor)	Rules (Claude Code)
Load on-demand Reusable workflows	Always loaded Quick guidelines	Always loaded Project context
<code>.claude/skills/</code> <code>.cursor/skills/</code>	<code>.cursor/rules/</code> Multiple <code>.md</code> files	<code>CLAUDE.md</code> Single markdown file
SKILL.md + frontmatter Scripts & assets	Markdown + frontmatter Text instructions	Simple markdown Architecture & standards

When to Use Which?

Use Skills For:

- Complex, multi-step workflows
- Tasks requiring scripts
- Domain-specific knowledge
- Reusable capabilities
- When context efficiency matters

Use Rules For:

- Quick style guidelines
- Project conventions
- Simple, persistent instructions
- Team-wide standards
- When you want always-on behavior

Importing Rules

Remote Rules (via GitHub):

- 1 Open Cursor Settings → Rules
- 2 Click **+ Add Rule** next to Project Rules
- 3 Select **Remote Rule (Github)**
- 4 Paste the GitHub repository URL

Agent Skills Integration:

- Cursor can load rules from Agent Skills standard
- Imported skills are treated as agent-decided rules
- Toggle on/off in Settings → Rules → Import Settings

Syncing

Imported rules stay synced with their source repository!

What is CLAUDE.md?

Definition

CLAUDE.md is Claude Code's rule definition file. It provides project-specific context that is loaded into every conversation.

Equivalent Concepts:

Claude Code	Cursor
CLAUDE.md	.cursor/rules/*.md
~/.claude/CLAUDE.md	User Rules

File Locations:

- Repository root: CLAUDE.md (project-level, shared with team)
- Parent directories: For monorepo setups
- Home folder: ~/.claude/CLAUDE.md (global)

CLAUDE.md Structure (Part 1)

A typical CLAUDE.md file includes project context:

```
# Project Context
```

```
## About This Project
```

Brief description of what your project does.

```
## Key Directories
```

- app/models/ - database models
- app/api/ - route handlers
- app/core/ - configuration

```
## Standards
```

- Type hints required on all functions
- pytest for testing

CLAUDE.md Structure (Part 2)

Common commands and workflows:

Common Commands

```
make dev      # start dev server  
make test     # run tests
```

Workflow

- Run typecheck after making changes
- Prefer single tests over full suite
- Use conventional commit messages

Getting Started with /init

The /init Command:

- Analyzes your project automatically
- Generates a starter CLAUDE.md tailored to your codebase
- Detects build commands, test instructions, and coding patterns

Usage:

- `cd your-project`
- `claude`
- `/init`

Think of /init as a starting point, not a finished product. Review and refine based on your team's practices!

Structuring CLAUDE.md Effectively

1. Give Claude a Map

- Include project summary and directory structure
- Document key dependencies and architectural patterns
- Explain non-standard organizational choices

2. Connect to Tools

- Document custom tools and scripts
- Include usage examples and when to invoke them
- Mention MCP server configurations

3. Define Standard Workflows

- Specify testing requirements before changes
- Document commit message format
- Include approval steps if needed

Best Practices for CLAUDE.md

DO Include

- Bash commands Claude can't guess
- Code style rules that differ from defaults
- Testing instructions
- Architectural decisions
- Common gotchas

DON'T Include

- Things Claude can figure out from code
- Standard language conventions
- Detailed API docs (link instead)
- File-by-file descriptions
- Self-evident practices

Key Takeaways

- **Skills** are portable, progressive packages for extending AI agents
- **CLAUDE.md** is Claude Code's rule file (equivalent to Cursor's `.cursor/rules/`)
- **Skills** load on-demand, **CLAUDE.md** loads every session
- **Locations:**
 - Skills: `.claude/skills/` (project) or `~/.claude/skills/` (global)
 - Rules: `CLAUDE.md` (project) or `~/.claude/CLAUDE.md` (global)
- **Use** `/skill` to view available skills
- **Use** `/init` to generate starter CLAUDE.md files

Thank You!